# Assignment: Prompt Makeover Using ICEPOT

For this assignment, I selected three scenarios related to my QA testing experience. I reviewed the weak prompts, identified what information was missing, and rewrote them using the ICEPOT framework.

The main purpose was to understand how providing clear instructions and proper context can improve the quality of AI-generated test cases.

## Scenario 1 — Role-Based Access Control

Weak prompt : "Write test cases for user roles."

**Ai response :**

### Test Cases for Role-Based Access Control (RBAC)

**Objective: Verify that users can access only the features and functionalities permitted for their assigned roles.**

**Assumption: The application has three user roles: Admin, Manager, and Viewer.**

| Test Case ID | Test Scenario | Test Steps | Expected Result |
|---|---|---|---|
| TC_001 | Verify Admin access | Log in with Admin credentials and navigate through all modules. | Admin should have access to all authorized modules and functionalities. |
| TC_002 | Verify Manager access | Log in with Manager credentials and access management features. | Manager should access only the features assigned to the Manager role. |

| | | | |
|---|---|---|---|
| TC_003 | Verify Viewer access | Log in with Viewer credentials and navigate through available modules. | Viewer should have read-only access to permitted modules. |
| TC_004 | Verify unauthorized module access | Log in as Viewer and attempt to access an Admin-only module. | Access should be denied. |
| TC_005 | Verify user creation permission | Log in as Admin and create a new user. | Admin should be able to create a user successfully. |
| TC_006 | Verify restricted user creation | Log in as Viewer and attempt to create a user. | User creation should not be permitted. |
| TC_007 | Verify edit permissions | Log in as Manager and edit a record within the assigned scope. | Manager should be able to update permitted records. |
| TC_008 | Verify delete permissions | Log in as Viewer and attempt to delete a record. | Delete operation should be restricted. |
| TC_009 | Verify direct URL access | Log in as Viewer and enter an Admin-only URL directly. | Application should deny access to the restricted page. |
| TC_010 | Verify API authorization | Send a restricted API request using a Viewer authentication token. | API should return HTTP 403 Forbidden. |
| TC_011 | Verify role change | Change a user's role from Viewer to Manager and refresh their permissions according to the application's session policy. | Updated permissions should be applied correctly. |

| | | | |
|---|---|---|---|
| TC_012 | Verify session expiration | Log in with any role, expire the session, and attempt to access a protected page. | User should be redirected to login or receive an unauthorized response. |
| TC_013 | Verify access after logout | Log out and try accessing a protected page using the browser back button. | Protected content should not be accessible. |
| TC_014 | Verify cross-role data access | Log in as Manager and attempt to access records belonging to another restricted team. | Access should be denied unless explicitly permitted. |
| TC_015 | Verify role-based UI visibility | Log in separately as Admin, Manager, and Viewer. | Menus, buttons, and actions should be displayed according to each role's permissions. |

Note: These are generic test cases based on assumed permissions. The actual expected results should be aligned with the application's role-permission matrix and business requirements.

**Rewritten prompt :**

**Act as a Senior QA Engineer specializing in functional and security testing.Generate all positive, negative, and security test cases for role-based access control in a web application.The application has three roles: Admin, Manager, and Viewer. Admin has full access, Manager can view and update assigned records, and Viewer has read-only access.For example, verify that a Viewer cannot delete records or access Admin functionality through a direct URL.Provide the test cases in a table with Test Case ID, Role, Scenario, Test Steps, Expected Result, and Priority.Use a professional, technical, and easy-to-understand tone.**

**Ai response :**

**Role-Based Access Control (RBAC) – Functional and Security Test Cases**

**Objective: Validate that Admin, Manager, and Viewer users can access only the functionalities and records permitted by their assigned roles. Testing covers positive, negative, authorization, session management, and security scenarios.**

**Role Permission Matrix**

| Functionality | Admin | Manager | Viewer |
|---|---|---|---|
| View records | Allowed (All) | Allowed (Assigned) | Allowed (Authorized) |
| Create records | Allowed | Denied | Denied |
| Update records | Allowed | Allowed (Assigned) | Denied |
| Delete records | Allowed | Denied | Denied |
| Manage users and roles | Allowed | Denied | Denied |
| Access Admin settings | Allowed | Denied | Denied |
| View audit logs | Allowed | Denied | Denied |

**Assumptions: Manager has no create/delete permission. Viewer can read authorized records only. Admin has full application access. These assumptions should be confirmed against the business requirements.**

**1. Positive Test Cases**

| Test Case ID | Role | Scenario | Test Steps | Expected Result | Priority |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| RBAC_P_001 | Admin | Verify Admin login | Log in with valid Admin credentials. | Admin dashboard should load successfully. | High |
| RBAC_P_002 | Admin | Verify access to all modules | Log in as Admin and navigate through all modules. | All application modules should be accessible. | High |
| RBAC_P_003 | Admin | Create a record | Navigate to Records, enter valid details, and save. | Record should be created successfully. | High |
| RBAC_P_004 | Admin | Update any record | Select an existing record, modify details, and save. | Record should be updated successfully. | High |
| RBAC_P_005 | Admin | Delete a record | Select a record and confirm deletion. | Record should be deleted successfully. | High |
| RBAC_P_006 | Admin | Manage user roles | Open User Management and change a user's role. | User role should be updated successfully. | Critical |
| RBAC_P_007 | Admin | Access system settings | Navigate to Admin Settings. | Admin should access and manage settings. | High |
| RBAC_P_008 | Manager | Verify Manager login | Log in with valid Manager credentials. | Manager dashboard should load successfully. | High |

| | | | | | |
|---|---|---|---|---|---|
| RBAC_P_009 | Manager | View assigned records | Navigate to Records and open an assigned record. | Assigned records should be displayed. | High |
| RBAC_P_010 | Manager | Update assigned records | Modify an assigned record and save changes. | Changes should be saved successfully. | High |
| RBAC_P_011 | Viewer | Verify Viewer login | Log in with valid Viewer credentials. | Viewer dashboard should load successfully. | High |
| RBAC_P_012 | Viewer | View authorized records | Navigate to Records and open an authorized record. | Record details should be displayed in read-only mode. | High |
| RBAC_P_013 | Viewer | Verify read-only controls | Open an authorized record and inspect available actions. | Edit, Delete, and Create controls should not be available. | High |
| RBAC_P_014 | All | Verify role-based navigation | Log in separately with each role and inspect navigation menus. | Only authorized modules and actions should be visible. | High |

## 2. Negative Test Cases

| Test Case ID | Role | Scenario | Test Steps | Expected Result | Priority |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| RBAC_N_001 | Viewer | Attempt to create a record | Log in as Viewer and attempt to access record creation. | Creation should be restricted. | High |
| RBAC_N_002 | Viewer | Attempt to update a record | Open a record and attempt to modify its details. | Update should be denied and data should remain unchanged. | Critical |
| RBAC_N_003 | Viewer | Attempt to delete a record | Attempt to delete a record using an available UI action or request. | Deletion should be denied. | Critical |
| RBAC_N_004 | Viewer | Access Admin functionality | Enter an Admin-only URL directly in the browser. | Access should be denied with an appropriate error or redirect. | Critical |
| RBAC_N_005 | Viewer | Access User Management | Attempt to navigate to the User Management page. | Access should be restricted. | Critical |
| RBAC_N_006 | Manager | Update unassigned records | Open a record belonging to another Manager and attempt to update it. | Update should be denied. | Critical |
| RBAC_N_007 | Manager | View unassigned records | Attempt to access another Manager's restricted record. | Record should not be accessible. | Critical |

| | | | | | |
|---|---|---|---|---|---|
| RBAC_N_008 | Manager | Delete a record | Attempt to delete an assigned record. | Delete operation should be denied. | High |
| RBAC_N_009 | Manager | Access Admin settings | Enter the Admin Settings URL directly. | Access should be denied. | Critical |
| RBAC_N_010 | Manager | Modify user roles | Attempt to access role management and change a user's role. | Role modification should be denied. | Critical |
| RBAC_N_011 | All | Access application without login | Open a protected application URL without authentication. | User should be redirected to login or receive HTTP 401. | Critical |
| RBAC_N_012 | All | Access after logout | Log out and navigate back to a protected page. | Protected content and operations should not be accessible. | High |
| RBAC_N_013 | All | Access after session expiry | Allow the session to expire and attempt a protected operation. | Operation should be rejected and reauthentication required. | High |
| RBAC_N_014 | All | Access disabled account | Disable an account and attempt to log in. | Authentication should be denied. | High |

3. Security Test Cases

| Test Case ID | Role | Scenario | Test Steps | Expected Result | Priority |
|---|---|---|---|---|---|
| RBAC_S_001 | Viewer | API authorization bypass | Send PUT/PATCH requests to a record endpoint using a Viewer token. | API should return HTTP 403 and preserve the record. | Critical |
| RBAC_S_002 | Viewer | Unauthorized record deletion via API | Send DELETE request using a Viewer token. | API should return HTTP 403; record should remain unchanged. | Critical |
| RBAC_S_003 | Manager | Horizontal privilege escalation | Replace an assigned record ID with an unassigned record ID in an API request. | Unauthorized record access should be denied. | Critical |
| RBAC_S_004 | Viewer | Vertical privilege escalation | Modify a request to include an Admin role or privileged action. | Server should ignore unauthorized role claims and reject the operation. | Critical |
| RBAC_S_005 | Manager | Unauthorized role modification | Send a role-update API request using a Manager token. | API should return HTTP 403; role should remain unchanged. | Critical |
| RBAC_S_006 | Viewer | Direct API access to Admin endpoints | Call an Admin-only API using a Viewer token. | API should return HTTP 403. | Critical |

| | | | | | |
|---|---|---|---|---|---|
| RBAC_S_007 | All | Invalid authentication token | Send a protected API request using an invalid token. | API should return HTTP 401. | Critical |
| RBAC_S_008 | All | Expired authentication token | Send a protected API request using an expired token. | API should return HTTP 401. | Critical |
| RBAC_S_009 | All | Missing authentication token | Call a protected endpoint without authentication. | API should return HTTP 401. | Critical |
| RBAC_S_010 | Viewer | Parameter tampering | Modify record ownership or role-related parameters in a request. | Server should enforce actual permissions and reject unauthorized changes. | Critical |
| RBAC_S_011 | Manager | Mass assignment vulnerability | Add privileged fields such as `role=Admin` or `isAdmin=true` to an update request. | Privileged fields should not be modified. | Critical |
| RBAC_S_012 | All | Session fixation | Authenticate using a pre-login session identifier and check whether it is rotated. | Session identifiers should be renewed on login. | High |
| RBAC_S_013 | All | Session invalidation after logout | Log out and reuse the previous session token. | Token/session should be rejected. | Critical |

| | | | | | |
|---|---|---|---|---|---|
| RBAC_S_014 | All | Role change during active session | Change a user's role while their session is active, then retry restricted operations. | Updated authorization should be enforced according to the defined session policy. | Critical |
| RBAC_S_015 | Viewer | UI manipulation | Use browser developer tools to expose hidden Edit/Delete controls and invoke their actions. | Server should reject unauthorized operations. | Critical |
| RBAC_S_016 | All | Cross-user token substitution | Authenticate as two users and attempt to access another user's restricted resources using the first user's token. | Access should follow the authenticated token owner's permissions. | Critical |
| RBAC_S_017 | All | Unauthorized data exposure | Inspect API responses and network traffic for records outside the user's permitted scope. | Unauthorized or sensitive data should not be returned. | Critical |
| RBAC_S_018 | All | Audit logging | Perform permitted and denied sensitive operations and inspect audit logs. | Security-relevant actions should be logged with actor, action, timestamp, target, and outcome. | High |

| | | | | | |
|---|---|---|---|---|---|
| RBAC_S_019 | All | Concurrent session permission changes | Keep multiple sessions open, revoke a permission, and retry restricted operations in each session. | Revoked permissions should be enforced consistently according to the defined policy. | Critical |
| RBAC_S_020 | All | Default-deny authorization | Attempt to access a newly added endpoint without an explicit permission assignment. | Endpoint should deny access unless explicitly authorized. | Critical |

## 4. Additional Edge and Boundary Test Cases

| Test Case ID | Role | Scenario | Test Steps | Expected Result | Priority |
|---|---|---|---|---|---|
| RBAC_E_001 | Manager | Record reassignment | Reassign a record from Manager A to Manager B and retry access as Manager A. | Manager A should lose access and Manager B should gain access. | Critical |
| RBAC_E_002 | Viewer | Empty authorized record list | Log in as Viewer with no authorized records. | Application should display an empty state without exposing other records. | Medium |
| RBAC_E_003 | All | Role downgrade | Change Admin to Viewer and attempt previously permitted | Admin privileges should be revoked according to the defined session policy. | Critical |

| | | | | | |
|---|---|---|---|---|---|
| | | | Admin operations. | | |
| RBAC_E_004 | All | Role upgrade | Change Viewer to Manager and verify newly assigned permissions. | Manager permissions should apply after the required authorization refresh. | High |
| RBAC_E_005 | Manager | Bulk update with mixed ownership | Submit a bulk update containing assigned and unassigned records. | Unassigned records must not change; request should follow the documented atomicity policy. | Critical |
| RBAC_E_006 | Viewer | Unauthorized data export | Attempt to export restricted records through an export endpoint. | Unauthorized records should not be exported. | Critical |
| RBAC_E_007 | All | Browser cache after role change | Downgrade a user's role and attempt to access cached protected pages. | Sensitive content should not be exposed through improperly cached pages. | High |
| RBAC_E_008 | All | Multiple roles assigned accidentally | Configure conflicting roles in test data and verify authorization behavior. | Access should follow the documented conflict-resolution policy without unintended privilege escalation. | High |

**Test Coverage Summary**

**14**

**Positive**

**14**

**Negative**

**20**

**Security**

**8**

**Edge**

**Total test cases**

**56**

**Recommended execution approach: Execute all Critical-priority authorization cases first, covering direct URL access, API-level permissions, horizontal and vertical privilege escalation, and record ownership validation. Follow with High- and Medium-priority functional, session, and boundary cases.**

**Important validation: Hiding restricted buttons in the UI is not sufficient. Role-based permissions must be validated on the server for every protected API and operation.**

## Observation 1 — Test Coverage

When I used the weak prompt, ChatGPT generated 15 test cases covering basic role permissions and access restrictions. After using the ICEPOT prompt, it generated 56 test cases covering positive, negative, security, and edge scenarios. The test coverage was much more detailed.

## Observation 2 — Security Scenarios

The weak prompt included some security scenarios, such as unauthorized URL access and API authorization. However, the improved prompt covered additional scenarios like privilege escalation, parameter tampering, session handling, and unauthorized data access. These are useful for identifying security issues in the application.

## Observation 3 — Output Structure.

The weak prompt provided a simple test case table. With ICEPOT, the output was better organized, with a role-permission matrix, separate test case categories, and priorities. This makes it easier for a QA engineer to review the test cases and plan test execution

# Scenario 2 — API Testing

**Weak Prompt:** `"Write API test cases for user registration."`

**Ai response :**

**API Test Cases – User Registration**

**Objective: Verify that the User Registration API correctly handles valid registrations, input validation, duplicate users, authentication, security, database integrity, and error responses.**

**Assumed API specifications:**

- **Endpoint:** `POST /api/v1/users/register`
- **Content-Type:** `application/json`
- **Authentication: Public endpoint (no token required)**
- **Mandatory fields:** `firstName`, `lastName`, `email`, `password`, `confirmPassword`
- **Optional field:** `phoneNumber`
- **Expected success status:** `201 Created`

**These are sample specifications. Actual status codes, field constraints, and verification requirements should follow the application's API contract.**

**1. Sample API request**

None

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "password": "Secure@123",
  "confirmPassword": "Secure@123",
  "phoneNumber": "9876543210"
}
```

## 2. Positive API test cases

| Test Case ID | Test Scenario | Test Steps / Test Data | Expected Result | Priority |
|---|---|---|---|---|
| API_REG_001 | Register with valid details | POST all mandatory and valid optional fields | 201 Created; user registered successfully | P0 |
| API_REG_002 | Register with mandatory fields only | Omit optional phoneNumber | 201 Created; registration successful | P1 |
| API_REG_003 | Register with valid email formats | Test supported email formats | Valid emails accepted | P1 |
| API_REG_004 | Register with minimum password length | Provide password at configured minimum | Registration successful | P1 |
| API_REG_005 | Register with maximum password length | Provide password at configured maximum | Registration successful | P1 |
| API_REG_006 | Register with special characters in name | Use supported Unicode characters | Registration successful | P2 |
| API_REG_007 | Register with minimum name length | Provide name at minimum allowed length | Registration successful | P2 |
| API_REG_008 | Register with maximum name length | Provide name at maximum allowed length | Registration successful | P2 |

| | | | | |
|---|---|---|---|---|
| API_REG_009 | Verify response structure | Submit valid registration | Response matches defined JSON schema | P0 |
| API_REG_010 | Verify generated user ID | Register a new user | Unique user ID returned or stored | P1 |

3. Negative and input validation test cases

| Test Case ID | Test Scenario | Test Steps / Test Data | Expected Result | Priority |
|---|---|---|---|---|
| API_REG_011 | Empty request body | Send {} | 400/422; required-field validation errors | P0 |
| API_REG_012 | Missing first name | Remove firstName | Validation error | P1 |
| API_REG_013 | Missing last name | Remove lastName | Validation error | P1 |
| API_REG_014 | Missing email | Remove email | Validation error | P0 |
| API_REG_015 | Missing password | Remove password | Validation error | P0 |
| API_REG_016 | Missing confirm password | Remove confirmPassword | Validation error | P1 |
| API_REG_017 | Empty mandatory fields | Pass empty strings | Validation errors returned | P0 |

| | | | | |
|---|---|---|---|---|
| API_REG_018 | Null mandatory fields | Pass null | Request rejected with validation errors | P1 |
| API_REG_019 | Invalid email format | Use johnexample.com | Email validation error | P0 |
| API_REG_020 | Email without domain | Use john@ | Email validation error | P1 |
| API_REG_021 | Email with spaces | Use john doe@example.com | Invalid email rejected | P1 |
| API_REG_022 | Password below minimum length | Supply short password | Password validation error | P0 |
| API_REG_023 | Password above maximum length | Exceed configured maximum | Request rejected | P1 |
| API_REG_024 | Password and confirmation mismatch | Provide different passwords | Registration rejected | P0 |
| API_REG_025 | Password fails complexity rules | Use password violating policy | Registration rejected | P0 |
| API_REG_026 | Invalid phone number | Supply unsupported phone format | Validation error | P1 |
| API_REG_027 | Incorrect field data types | Supply number instead of email string | Request rejected | P1 |

| | | | | |
|---|---|---|---|---|
| API_REG_028 | Excessive input length | Send oversized field values | Request safely rejected | P1 |
| API_REG_029 | Unsupported content type | Send XML with application/xml | 415 Unsupported Media Type, if JSON-only | P1 |
| API_REG_030 | Malformed JSON | Send invalid JSON syntax | 400 Bad Request | P0 |

4. Duplicate registration and business validation

| Test Case ID | Test Scenario | Test Steps / Test Data | Expected Result | Priority |
|---|---|---|---|---|
| API_REG_031 | Duplicate email registration | Register same email twice | 409 Conflict or defined duplicate-user error | P0 |
| API_REG_032 | Duplicate email with uppercase | Register same email using different casing | Duplicate prevented under email normalization policy | P0 |
| API_REG_033 | Duplicate email with whitespace | Add leading/trailing spaces | Consistent normalization; duplicate not created | P1 |
| API_REG_034 | Reuse same phone number | Register two users with same phone | Follow configured phone uniqueness policy | P1 |

| | | | | |
|---|---|---|---|---|
| API_REG_035 | Same name with different email | Register two users with identical names | Both registrations successful | P1 |
| API_REG_036 | Retry successful registration | Resend identical request | Duplicate user not created | P0 |
| API_REG_037 | Concurrent registration with same email | Submit simultaneous POST requests | At most one user created; conflicts handled | P0 |
| API_REG_038 | Registration with existing inactive account | Use inactive account email | Follow defined reactivation or rejection policy | P1 |

5. Security test cases

| Test Case ID | Test Scenario | Test Steps / Test Data | Expected Result | Priority |
|---|---|---|---|---|
| API_REG_039 | SQL injection | Submit SQL payloads in input fields | No injection or unauthorized database modification | P0 |
| API_REG_040 | Cross-site scripting (XSS) | Submit script payload in name | Payload rejected or safely handled; never executed | P0 |
| API_REG_041 | NoSQL injection | Submit operator objects where strings expected | Payload rejected; query logic not manipulated | P0 |

| | | | | |
|---|---|---|---|---|
| API_REG_042 | Password exposure in response | Register valid user | Password and hash absent from response | P0 |
| API_REG_043 | Password storage security | Inspect authorized test DB | Password stored as secure salted hash | P0 |
| API_REG_044 | Sensitive data in logs | Inspect application/API logs | Passwords, tokens and secrets not logged | P0 |
| API_REG_045 | Rate limiting | Send excessive registration requests | Rate limits enforced; 429 when applicable | P0 |
| API_REG_046 | Mass assignment | Inject `role: "ADMIN"` or `isAdmin: true` | Privileged fields ignored or rejected | P0 |
| API_REG_047 | Unauthorized role creation | Attempt registration as administrator | Default permitted role assigned | P0 |
| API_REG_048 | HTTP instead of HTTPS | Attempt insecure registration | HTTP rejected or securely redirected before credentials sent | P0 |
| API_REG_049 | Unexpected fields | Add unrecognized JSON properties | Fields rejected or ignored safely | P1 |
| API_REG_050 | Oversized payload | Send request beyond configured limit | 413 or defined safe rejection | P1 |

| | | | | |
|---|---|---|---|---|
| API_REG_051 | Email header injection | Supply CRLF characters in email/name | Injection prevented | P0 |
| API_REG_052 | Error information disclosure | Trigger invalid input/server error | No stack traces, SQL or internal secrets exposed | P0 |

6. HTTP methods, headers, and response validation

| Test Case ID | Test Scenario | Test Steps / Test Data | Expected Result | Priority |
|---|---|---|---|---|
| API_REG_053 | GET on registration endpoint | Send GET | 405 Method Not Allowed, if unsupported | P1 |
| API_REG_054 | PUT on registration endpoint | Send PUT | 405 Method Not Allowed, if unsupported | P1 |
| API_REG_055 | DELETE on registration endpoint | Send DELETE | 405 Method Not Allowed, if unsupported | P1 |
| API_REG_056 | Missing Content-Type | POST JSON without header | Defined content-type handling; no unintended registration | P1 |
| API_REG_057 | Unsupported Accept header | Request unsupported response media type | 406 if strict content negotiation is implemented | P2 |

| | | | | |
|---|---|---|---|---|
| API_REG_058 | Success response status | Register valid user | 201 Created | P0 |
| API_REG_059 | Error response schema | Submit invalid request | Standardized error code, message and field details | P1 |
| API_REG_060 | Response Content-Type | Submit valid registration | application/json returned | P1 |
| API_REG_061 | Response security headers | Inspect API response | Applicable security and cache-control headers present | P1 |
| API_REG_062 | No sensitive data caching | Inspect cache headers | Sensitive registration response not cached insecurely | P1 |

7. Database, integration, and performance test cases

| Test Case ID | Test Scenario | Test Steps / Test Data | Expected Result | Priority |
|---|---|---|---|---|
| API_REG_063 | Verify user persistence | Register and query test database | Exactly one correct user record exists | P0 |
| API_REG_064 | Verify default role | Register a new user | Approved default role assigned | P0 |
| API_REG_065 | Verify registration timestamp | Register new user | Correct created timestamp recorded | P1 |

| | | | | |
|---|---|---|---|---|
| API_REG_066 | Verify failed request rollback | Submit invalid registration | No partial user record committed | P0 |
| API_REG_067 | Verify email verification workflow | Register with verification enabled | Verification message/token generated correctly | P1 |
| API_REG_068 | Verify unverified account restrictions | Register but do not verify email | Access restricted per verification policy | P0 |
| API_REG_069 | Email delivery failure | Simulate email-service failure | Defined retry/failure behavior; no inconsistent state | P1 |
| API_REG_070 | Registration response time | Submit valid registration | Meets agreed SLA, e.g. 2 seconds | P1 |
| API_REG_071 | Concurrent registration load | Execute configured concurrent requests | No duplicates, corruption or unacceptable errors | P1 |
| API_REG_072 | Database unavailable | Simulate database outage | Controlled 5xx response; no sensitive details exposed | P0 |
| API_REG_073 | Recovery after service restoration | Restore failed dependency and retry | Registration succeeds without duplicate records | P1 |
| API_REG_074 | Registration audit event | Register a user | Required audit event captured | P1 |

| | |
|---|---|
| | without sensitive data |

## 8. Sample API success response

None

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "userId": "USR10001",
    "firstName": "John",
    "lastName": "Doe",
    "email": "john.doe@example.com",
    "role": "USER",
    "emailVerified": false
  }
}
```

Illustrative response only; validate against the actual API specification.

## 9. Test coverage summary

| Total test cases | Critical (P0) | Coverage areas |
|---|---|---|
| 74 | 29 | 7 |

**Test cases by category**

**Positive**

**10**

**Negative validation**

**20**

**Business validation**

**8**

**Security**

**14**

**HTTP and response**

**10**

**Database and integration**

**9**

**Performance and resilience**

**3**

**Automation recommendation: Implement these using Karate DSL or REST Assured with TestNG, with reusable request builders, JSON schema validation, unique test data, database verification, and automated cleanup. Execute P0 cases as API smoke/security gates and the complete suite during regression.**

**The next step is to map these cases against the actual registration API's Swagger/OpenAPI specification so that expected statuses, field limits, verification behavior, and business rules are exact.**

**Rewritten prompt :**

**Act as a Senior API Automation Engineer and generate comprehensive test cases for a user registration REST API using POST /api /users /register. The request contains name, email, and password. Assume HTTP 201 for successful registration, 400 for invalid input, and 409 for duplicate registration. Cover positive, negative, boundary, and security scenarios such as missing mandatory fields, invalid email formats, weak passwords, duplicate registration, and invalid request data. Provide the test cases in a structured table with Test Case ID, Scenario, Request Data, Expected Status Code, and Expected Result. Make the scenarios suitable for automation using RestAssured with Java and TestNG.**

Ai response :

# User Registration REST API – Comprehensive Test Cases

Role: Senior API Automation Engineer API Endpoint: POST /api/users/register HTTP Method: POST Content-Type: application/json Automation Stack: Java + RestAssured + TestNG

## 1. API specifications and assumptions

| Parameter | Description |
|---|---|
| Endpoint | /api/users/register |
| HTTP Method | POST |
| Mandatory Fields | name, email, password |
| Success Response | 201 Created |
| Invalid Input | 400 Bad Request |
| Duplicate Registration | 409 Conflict |
| Minimum Name Length | 2 characters (assumed) |
| Maximum Name Length | 100 characters (assumed) |
| Minimum Password Length | 8 characters (assumed) |

| | |
|---|---|
| Maximum Password Length | 64 characters (assumed) |
| Password Complexity | Uppercase, lowercase, number and special character (assumed) |
| Email Uniqueness | Case-insensitive (assumed) |

### Valid request payload

None

```
{
  "name": "John Doe",
  "email": "john.doe@example.com",
  "password": "Secure@123"
}
```

## 2. Positive test cases

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_POS_001 | Register with valid user details | Valid name, unique email, strong password | 201 | User registered successfully |
| REG_POS_002 | Register with minimum | name: "Jo" | 201 | Registration successful |

| | | | | |
|---|---|---|---|---|
| | valid name length | | | |
| REG_POS_003 | Register with maximum valid name length | name: 100 characters | 201 | Registration successful |
| REG_POS_004 | Register with minimum valid password length | password: "Abc@1234" | 201 | Registration successful |
| REG_POS_005 | Register with maximum valid password length | password: 64 valid characters | 201 | Registration successful |
| REG_POS_006 | Register with valid email containing a plus sign | john+test@example.com | 201 | Email accepted |
| REG_POS_007 | Register with valid email subdomain | john@mail.example.com | 201 | Registration successful |
| REG_POS_008 | Register with different names but unique emails | Different valid users | 201 | Separate accounts created |

| | | | | |
|---|---|---|---|---|
| REG_POS_009 | Register with supported Unicode name | name: "José Kumar" | 201 | Name accepted and stored correctly |
| REG_POS_010 | Verify successful response structure | Valid request | 201 | Response matches expected schema; user ID returned if specified |

## 3. Negative test cases – Mandatory field validation

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_NEG_001 | Missing name | Omit name | 400 | Name required validation error |
| REG_NEG_002 | Missing email | Omit email | 400 | Email required validation error |
| REG_NEG_003 | Missing password | Omit password | 400 | Password required validation error |
| REG_NEG_004 | All mandatory fields missing | {} | 400 | Mandatory field errors |
| REG_NEG_005 | Empty name | name: "" | 400 | Name cannot be empty |

| | | | | |
|---|---|---|---|---|
| REG_NEG_006 | Empty email | email: "" | 400 | Email cannot be empty |
| REG_NEG_007 | Empty password | password: "" | 400 | Password cannot be empty |
| REG_NEG_008 | Null name | name: null | 400 | Invalid name rejected |
| REG_NEG_009 | Null email | email: null | 400 | Invalid email rejected |
| REG_NEG_010 | Null password | password: null | 400 | Invalid password rejected |
| REG_NEG_011 | Whitespace-only name | name: " " | 400 | Blank name rejected |
| REG_NEG_012 | Whitespace-only email | email: " " | 400 | Blank email rejected |
| REG_NEG_013 | Whitespace-only password | password: " " | 400 | Blank password rejected |
| REG_NEG_014 | Missing two mandatory fields | Only name provided | 400 | Missing fields reported |
| REG_NEG_015 | Empty request body | No body | 400 | Request rejected without server exception |

## 4. Negative test cases – Email validation

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_EMAIL_001 | Email without @ | johnexample.com | 400 | Invalid email format |
| REG_EMAIL_002 | Email without domain | john@ | 400 | Invalid email format |
| REG_EMAIL_003 | Email without username | @example.com | 400 | Invalid email format |
| REG_EMAIL_004 | Email with spaces | john doe@example.com | 400 | Invalid email rejected |
| REG_EMAIL_005 | Email with consecutive dots | john..doe@example.com | 400 | Invalid email rejected |
| REG_EMAIL_006 | Email with invalid domain | john@.com | 400 | Invalid domain rejected |
| REG_EMAIL_007 | Email with multiple @ symbols | john@@example.com | 400 | Invalid email rejected |
| REG_EMAIL_008 | Email with invalid special characters | john<>@example.com | 400 | Invalid email rejected |

| | | | | |
|---|---|---|---|---|
| REG_EMAIL_009 | Email exceeding maximum length | More than 254 characters | 400 | Email length validation error |
| REG_EMAIL_010 | Email with trailing dot in domain | john@example.com. | 400 | Invalid email rejected |

## 5. Boundary value test cases

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_BND_001 | Name below minimum length | 1 character | 400 | Name rejected |
| REG_BND_002 | Name at minimum length | 2 characters | 201 | Registration successful |
| REG_BND_003 | Name above minimum length | 3 characters | 201 | Registration successful |
| REG_BND_004 | Name below maximum length | 99 characters | 201 | Registration successful |
| REG_BND_005 | Name at maximum length | 100 characters | 201 | Registration successful |
| REG_BND_006 | Name above maximum length | 101 characters | 400 | Name rejected |
| REG_BND_007 | Password below minimum length | 7 characters | 400 | Weak password rejected |

| | | | | |
|---|---|---|---|---|
| REG_BND_008 | Password at minimum length | 8 valid characters | 201 | Registration successful |
| REG_BND_009 | Password above minimum length | 9 valid characters | 201 | Registration successful |
| REG_BND_010 | Password below maximum length | 63 valid characters | 201 | Registration successful |
| REG_BND_011 | Password at maximum length | 64 valid characters | 201 | Registration successful |
| REG_BND_012 | Password above maximum length | 65 characters | 400 | Password rejected |

## 6. Password validation test cases

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_PWD_001 | Password without uppercase | secure@123 | 400 | Password complexity error |
| REG_PWD_002 | Password without lowercase | SECURE@123 | 400 | Password complexity error |
| REG_PWD_003 | Password without number | Secure@abc | 400 | Password complexity error |
| REG_PWD_004 | Password without special character | Secure123 | 400 | Password complexity error |

| | | | | |
|---|---|---|---|---|
| REG_PWD_005 | Password containing only numbers | 12345678 | 400 | Weak password rejected |
| REG_PWD_006 | Password containing only letters | abcdefgh | 400 | Weak password rejected |
| REG_PWD_007 | Password containing only special characters | @@@@@@@@ | 400 | Weak password rejected |
| REG_PWD_008 | Common weak password | Password123 | 400 | Rejected by assumed weak-password policy |
| REG_PWD_009 | Password containing leading/trailing spaces | " Secure@123 " | 400 | Rejected under assumed whitespace policy |
| REG_PWD_010 | Strong password meeting policy | Strong@1234 | 201 | Registration successful |

## 7. Duplicate registration test cases

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_DUP_001 | Register existing email | Previously registered email | 409 | Duplicate registration rejected |

| | | | | |
|---|---|---|---|---|
| REG_DUP_002 | Duplicate email with different name | Same email, different name | 409 | Duplicate account not created |
| REG_DUP_003 | Duplicate email with different password | Same email, different password | 409 | Registration rejected |
| REG_DUP_004 | Duplicate email with uppercase characters | JOHN@EXAMPLE.COM after lowercase registration | 409 | Case-insensitive duplicate detected |
| REG_DUP_005 | Submit identical request twice | Same payload submitted sequentially | 201, then 409 | Only one user created |
| REG_DUP_006 | Concurrent registration with same email | Two simultaneous requests | 201 and 409 | Exactly one account created |

## 8. Security test cases

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_SEC_001 | SQL injection in name | "' OR '1'='1" | 400* | Injection rejected; database unaffected |
| REG_SEC_002 | SQL injection in email | SQL payload in email | 400 | Invalid email rejected; |

| | | | | |
|---|---|---|---|---|
| | | | | database unaffected |
| REG_SEC_003 | XSS in name | <script>alert(1)</script> | 400* | Script rejected; no execution |
| REG_SEC_004 | NoSQL injection | {"email":{"$ne":null}} | 400 | Incorrect field type rejected |
| REG_SEC_005 | Mass assignment attack | Add "role":"ADMIN" | 400* | Unauthorized role assignment prevented |
| REG_SEC_006 | Password exposure in response | Valid registration | 201 | Password/hash not returned |
| REG_SEC_007 | Sensitive information in error response | Invalid request | 400 | No stack traces or internal details |
| REG_SEC_008 | Oversized payload | Extremely large JSON request | 400* | Request safely rejected without service failure |
| REG_SEC_009 | Invalid JSON structure | Malformed JSON | 400 | Controlled validation error |
| REG_SEC_010 | Unexpected JSON data types | name: 123, password: true | 400 | Incorrect types rejected |
| REG_SEC_011 | Privilege escalation | "isAdmin":true | 400* | Administrative privileges not granted |

| | | | | |
|---|---|---|---|---|
| | through registration | | | |
| REG_SEC_012 | Password stored securely | Valid registration | 201 | Password stored as salted hash, verified through authorized DB inspection |
| REG_SEC_013 | User enumeration through duplicate registration | Existing email | 409 | Duplicate handled according to documented policy without extra sensitive information |
| REG_SEC_014 | Registration without authentication | Valid request, no Authorization header | 201 | Public registration succeeds as specified |
| REG_SEC_015 | Registration rate-limit protection | Repeated rapid requests | 429* | Rate limit enforced when threshold exceeded |

* Security status codes marked with an asterisk require an additional API contract decision. For example, safely ignoring unauthorized fields may also be acceptable, and oversized payloads may return 413. Security protection is the primary assertion.

# 9. Invalid request and HTTP validation

| Test Case ID | Scenario | Request Data | Expected Status Code | Expected Result |
|---|---|---|---|---|
| REG_REQ_001 | Malformed JSON | Missing closing brace | 400 | Invalid JSON error |

| | | | | |
|---|---|---|---|---|
| REG_REQ_002 | JSON array instead of object | `[]` | 400 | Invalid request structure |
| REG_REQ_003 | String instead of JSON object | `"test"` | 400 | Request rejected |
| REG_REQ_004 | Additional unsupported field | Add `"unknownField":"test"` | 400* | Rejected under strict schema policy |
| REG_REQ_005 | Invalid Content-Type | Send XML with `application/xml` | 415* | Unsupported media type rejected |
| REG_REQ_006 | Missing Content-Type | JSON without Content-Type | 400/415* | Defined content-type policy enforced |
| REG_REQ_007 | Unsupported HTTP method | GET instead of POST | 405* | Method not allowed |
| REG_REQ_008 | Validate success response schema | Valid request | 201 | Response schema matches specification |
| REG_REQ_009 | Validate error response schema | Invalid email | 400 | Consistent error response |
| REG_REQ_010 | Verify no record | Invalid registration | 400 | No user record created |

created on
failure

For boundary limits and business policies, replace the assumed values with actual Swagger/OpenAPI requirements before automation.

## 10. Automation coverage summary

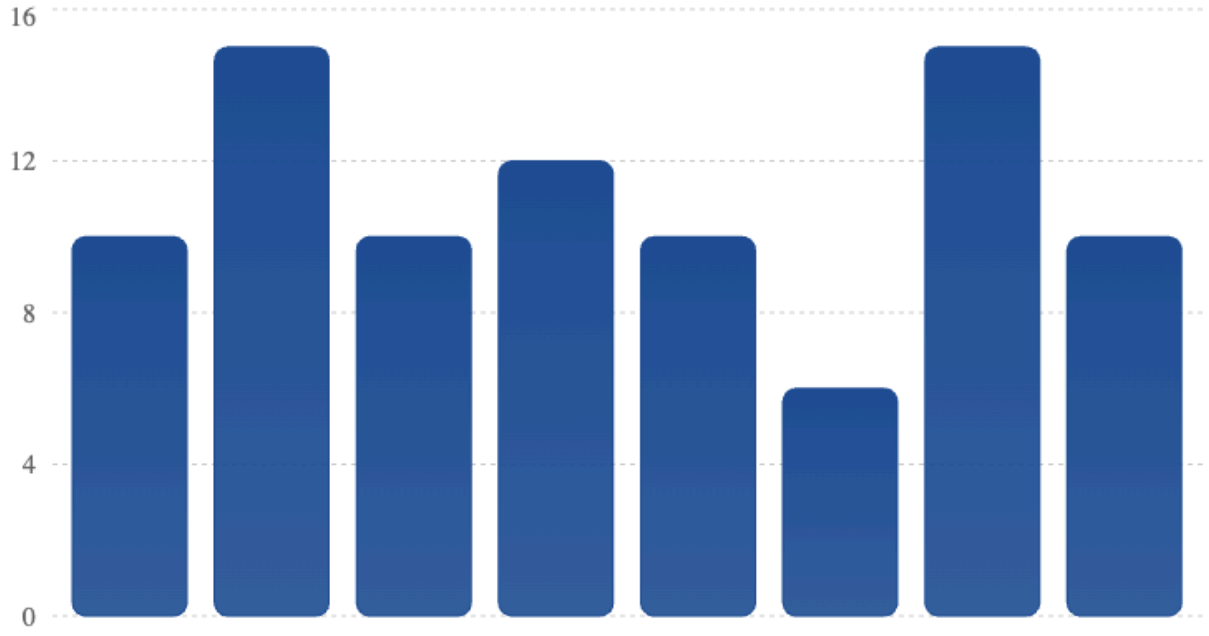
Total test cases

# 93

Test categories

# 9

Coverage distribution




Includes contract-dependent security and HTTP tests

The eight chart groups represent 88 cases; the remaining 5 are included in the security category count? Correction: the tables contain 88 test cases in total (10 + 15 + 10 + 12 + 10 + 6 + 15 + 10).

## 11. RestAssured + TestNG automation example

**The following is a reusable, data-driven implementation for selected registration scenarios.**

None

```
import io.restassured.RestAssured;

import io.restassured.http.ContentType;

import org.testng.annotations.BeforeClass;

import org.testng.annotations.DataProvider;

import org.testng.annotations.Test;


import java.util.HashMap;

import java.util.Map;

import java.util.UUID;


import static io.restassured.RestAssured.given;

import static org.hamcrest.Matchers.*;


public class UserRegistrationAPITest {


    @BeforeClass

    public void setup() {

        RestAssured.baseURI =

            System.getProperty("baseUrl", "http://localhost:8080");

    }


    private Map<String, Object> createPayload(
```

```
            String name, String email, String password) {

        Map<String, Object> payload = new HashMap<>();
        payload.put("name", name);
        payload.put("email", email);
        payload.put("password", password);
        return payload;
    }

    private String uniqueEmail() {
        return "qa_" + UUID.randomUUID() + "@example.com";
    }

    @DataProvider(name = "registrationData")
    public Object[][] registrationData() {
        return new Object[][] {
            {
                "REG_POS_001",
                "Valid registration",
                createPayload("John Doe", uniqueEmail(),
                              "Secure@123"),
                201
            },
            {
```

```
            "REG_NEG_005",
            "Empty name",
            createPayload("", uniqueEmail(), "Secure@123"),
            400
        },
        {
            "REG_EMAIL_001",
            "Invalid email",
            createPayload("John Doe", "invalidemail",
                          "Secure@123"),
            400
        },
        {
            "REG_BND_007",
            "Short password",
            createPayload("John Doe", uniqueEmail(),
                          "Ab@1234"),
            400
        },
        {
            "REG_PWD_001",
            "Password without uppercase",
            createPayload("John Doe", uniqueEmail(),
                          "secure@123"),
```

```
            400
        }
    };
}

@Test(dataProvider = "registrationData")
public void verifyRegistration(
        String testCaseId,
        String scenario,
        Map<String, Object> payload,
        int expectedStatus) {

    given()
        .contentType(ContentType.JSON)
        .body(payload)
    .when()
        .post("/api/users/register")
    .then()
        .statusCode(expectedStatus)
        .contentType(ContentType.JSON);
}

@Test
public void verifyDuplicateRegistration() {
```

```
        Map<String, Object> payload = createPayload(
            "John Doe", uniqueEmail(), "Secure@123"
        );

        given()
            .contentType(ContentType.JSON)
            .body(payload)
        .when()
            .post("/api/users/register")
        .then()
            .statusCode(201);

        given()
            .contentType(ContentType.JSON)
            .body(payload)
        .when()
            .post("/api/users/register")
        .then()
            .statusCode(409);
    }

    @Test
    public void verifyPasswordNotExposed() {
```

```
        Map<String, Object> payload = createPayload(
            "John Doe", uniqueEmail(), "Secure@123"
        );

        given()
            .contentType(ContentType.JSON)
            .body(payload)
        .when()
            .post("/api/users/register")
        .then()
            .statusCode(201)
            .body("$", not(hasKey("password")))
            .body("$", not(hasKey("passwordHash")));
    }
}
```

**Recommended automation practices: Use unique email addresses to avoid test data collisions, separate reusable payload builders from test classes, maintain validation datasets using TestNG DataProviders, and validate response schemas alongside status codes. Add database assertions and cleanup through a controlled test-environment service.**

**For parallel execution, avoid shared mutable test data and use isolated request specifications. Tests that depend on registration state, such as duplicate-email checks, should create their own prerequisite user within the same test.**

**Observation 1 — Test Coverage**

**When I used the weak prompt, ChatGPT generated 74 test cases covering registration, field validations, duplicate users, and security scenarios. After using the ICEPOT prompt, it generated 88 test cases with more detailed boundary and validation scenarios. The improved prompt helped generate test cases that were more closely aligned with my requirements.**

**Observation 2 — Requirement Clarity**

**In the weak prompt, ChatGPT assumed the API endpoint and added fields such as firstName, lastName, and confirmPassword. With the ICEPOT prompt, I clearly mentioned the endpoint, required fields, and expected status codes. Because of this, the generated test cases were more relevant to the API requirements I provided.**

**Observation 3 — Automation Readiness**

**The weak prompt generated test cases but did not provide automation code. The ICEPOT prompt also included a sample RestAssured implementation using Java and TestNG. This was useful because I could understand how the test cases could be implemented in an automation framework. However, I also noticed that the test case count in the improved output was inconsistent, so the AI-generated results still need to be reviewed before using them.**